



INDIA.RUNS

Build what next India runs on

Team Name : **Evolve**

Team Leader Name : **Rayyan Ahmed Shaikh**

Problem Statement : **Track 1 - The Data & AI Challenge: Intelligent Candidate Discovery & Ranking**

PARAKH (परख - to assay; to test whether gold is genuine) · Rayyan Ahmed Shaikh · Inchara P · Harshita Nagesh

Solution Overview

- **What is your proposed solution?**

Parakh - a ranker that verifies every profile before trusting a word of it. An Integrity Gate rejects the dataset's planted fakes outright; two frontier LLMs (DeepSeek-V4 + Qwen3-235B) grade every plausible candidate against the JD offline; a third model family (GLM-5.2) re-judges the top; only candidates all three families endorse reach the top-10. Every model call is pre-computed and cached - the submitted ranker runs on a laptop CPU in ~3 minutes, fully offline, and reproduces the CSV byte-for-byte.

- **What differentiates it from traditional candidate matching?**

Traditional matchers trust what candidates claim and count keyword overlap - and this dataset is engineered to kill exactly that (stuffed skill lists, ~80 planted honeypots, real gems hidden in plain language). Parakh ranks on verified evidence instead: what the career narrative proves, what 23 behavioral signals confirm, and what three independent AI judges agree on. Assay, don't match.

JD Understanding & Candidate Evaluation

- **Key requirements extracted from the JD**

Must-haves: production embedding retrieval (sentence-transformers / BGE / E5), vector-DB or hybrid-search operations (FAISS, Pinecone, Elasticsearch), rigorous ranking evaluation (NDCG, MRR, A/B), strong Python, 5-9 yrs (ideal 6-8) mostly at product companies, India-based or willing to relocate. Explicit disqualifiers the JD itself states: consulting-only careers, research without production, recent LangChain-only 'AI experience', CV/speech-only depth, title-chasers, unreachable candidates.

Which candidate signals matter most?

The skills array is the least trustworthy field - it is what stuffers stuff. We weight the career narrative (did they build and ship retrieval / ranking / recsys), title trajectory, product-vs-services history, and behavioral signals: recruiter response rate, last-active recency, notice period, open-to-work. A perfect profile that never replies is not a hire.

Evaluating fit beyond keyword matching

Free-text evidence extraction plus LLM judgment: 7,334 candidates describe real ranking work under non-ML titles, and keyword filters miss every one of them. Our judges read the story, not the tags - and a recall-rescue sweep re-checks all 100k for narrative evidence the rules under-ranked, which recovered 9 hidden gems.

Ranking Methodology

- **Retrieve, score, rank**

Stage 1 - Integrity Gate: reject impossible profiles ('expert' skills used 0 months, tenure longer than the whole career) and keyword-stuffers (AI skills with zero supporting narrative). Stage 2 - a transparent JD rubric scores all 100,000: role, narrative evidence, must-have tech, experience band, company type, location, availability. Stage 3 - offline judging: DeepSeek-V4 and Qwen3-235B each grade the rule-shortlist top-3,000 PLUS a 2,500-profile full-pool sweep; GLM-5.2 re-judges the top-100.

Models, algorithms, heuristics

Frontier open-weight LLMs as offline judges (via Nebius Token Factory), deterministic feature engineering, and narrative-evidence extraction. No fine-tuning required. The submitted ranker is pure Python reading cached judgments: fast, auditable, reproducible.

Combining signals into one ranking

final score = average judge tier (0-5) + rule score as the within-tier tiebreak, scaled to [0,1]. Honeypots are forced to the floor; equal scores tie-break by candidate_id ascending, exactly as the spec requires. Judgment dominates; rules order within agreement bands.

Design Rationale - Why Not Just Embeddings?

Where would the labels come from?

A learned ranker needs training labels - and this dataset ships none. The only way to get a supervision signal is to synthesize judgment, which is exactly what our offline teachers produce. Any 'embeddings + learned ranker' build starts by bootstrapping labels the same way we did; we simply spent the budget on making that judgment reliable (three model families) instead of hiding it inside a trained model.

The rules reward this design

Runtime is CPU-only, no network, five minutes. The challenge brief itself says: 'plan for a small ranker over precomputed features.' Parakh is that sentence taken seriously - think offline with frontier models once, serve cached judgment in milliseconds, reproduce byte-for-byte.

On this dataset, cosine similarity rewards the fraud

Keyword-stuffed profiles sit close to the JD in embedding space precisely because they copied its vocabulary - semantic similarity scores the disguise, not the work. Verified narrative evidence plus judge agreement does not. Embedding retrieval is on our roadmap for 200k+ scale - as a recall layer underneath the same Integrity Gate and judges, never as a replacement for them.

Explainability & Data Validation

- **How are ranking decisions explained?**

Every one of the 100 rows carries a 1-2 line justification citing real profile facts (title, years, named tech, response rate) tied to a JD requirement - with tone matched to rank: low ranks lead with the honest concern, not praise.

Preventing hallucinated justifications

Reasons are generated strictly from the profile JSON, then programmatically verified: every tech or company token quoted must literally exist in that profile; failures are regenerated or replaced with grounded fallbacks. 95 of 100 final reasons pass strict verification; the rest use verified teacher text.

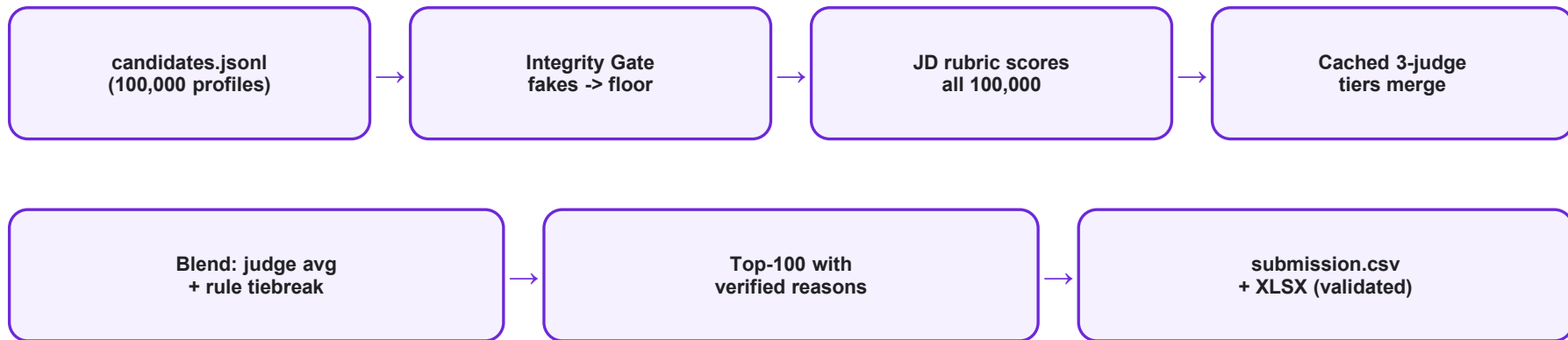
Handling inconsistent or suspicious profiles

That is the Integrity Gate's job: 42 hard honeypots detected analytically, zero in our top-100 (the disqualification line is 10%). Stuffers are suppressed multiplicatively. We also tested a tempting extra check (last-active before signup) and rejected it - it fires on 7,496 profiles of generator noise. Precision over paranoia.

End-to-End Workflow

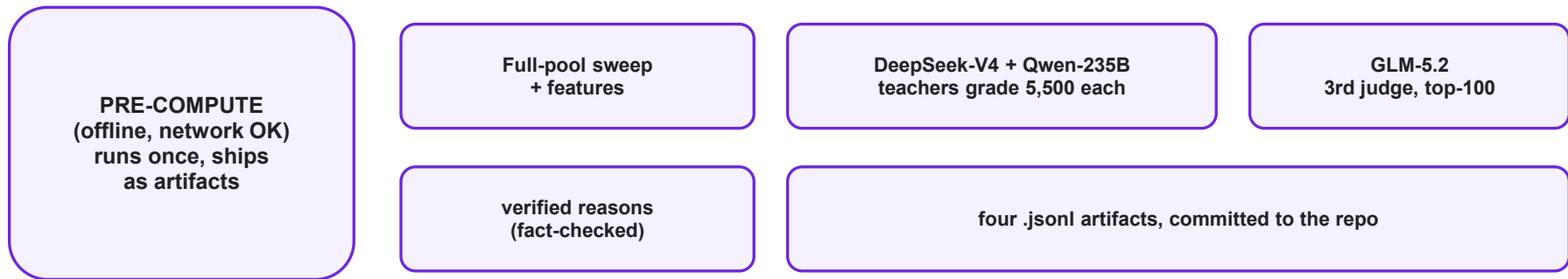
- **From JD input to ranked output - one command**

`python rank.py --candidates candidates.jsonl --out submission.csv` · ~3 min on a laptop CPU, no network, no GPU. The heavy thinking happened once, offline; artifacts ship with the repo.

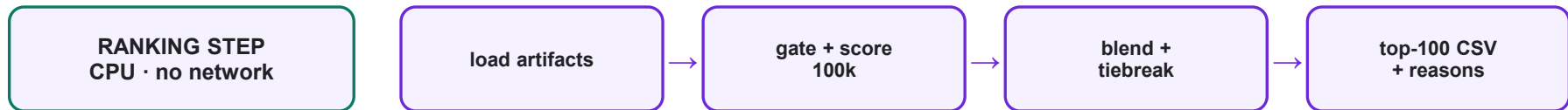


System Architecture

Two planes: the intelligence is pre-computed; the ranking step just reads it.



Self-test on every run: ~180-265 s wall (limit 300) · 0.21 GB RAM (limit 16) · bare command reproduces the submission byte-identically.



Results & Performance

- **Ranking quality**

Judged by an independent model that never touched the ranking (Llama-3.3-70B): NDCG@10 rose 0.64 (rules) -> 0.88 (+teacher) -> 1.00 (3-family ensemble); the final top-10 is unanimously tier-5 across DeepSeek, Qwen and GLM. Zero honeypots and zero stuffers in the top-100. The full-pool sweep rescued 9 genuine hidden gems the shortlist missed. Honest note: judge agreement is a strong proxy, not the organisers' hidden truth.

Runtime and compute constraints

Built-in self-test prints proof on every run: ~180-265 s wall against the 300 s limit, 0.21 GB RAM against 16 GB, CPU-only, zero network calls. The bare reproduce command regenerates the submitted CSV byte-for-byte from the repo.

1.00

NDCG@10 (independent judge)

10/10

top-10 tier-5, 3 model families

0

honeypots in our top-100

Technologies Used

- **Stack and why**

Ranker: Python 3.11, stdlib-first (orjson fast parse, openpyxl for XLSX) - deliberately dependency-light so Stage-3 reproduction cannot break. Offline judging: Nebius Token Factory serving DeepSeek-V4-Pro and Qwen3-235B as teachers, GLM-5.2 as third judge, Llama-3.3-70B as the independent evaluator - four different model families, chosen so that agreement actually means something. Sandbox: Gradio on Hugging Face Spaces, running the exact same code path. Git with full iteration history (12 commits of real evolution, not a final dump).

Submission Assets

Everything needed to verify us

GitHub (full code + artifacts)

github.com/TheClazer/parakh

Live sandbox (Hugging Face Space)

huggingface.co/spaces/TheClazer/parakh-ranker

Ranked output

Parakh_top100.xlsx / submission.csv - passes the official validator

Reproduce

```
python rank.py --candidates candidates.jsonl --out submission.csv
```

Contact

Rayyan Ahmed Shaikh · therayyn16@gmail.com · Team Evolve



INDIA.RUNS

Build what next India runs on

THANK YOU

PARAKH - assay, don't match · Team Evolve · thank you